

TDA (Abstract Data Type -Tipos de datos Abstractos -)

Es un tipo de dato especial creado por el programador a la medida de su necesidad. Este tipo de dato se compone a su vez de tipos de datos primitivos del lenguaje de programación, éstos pueden ser iguales o distintos. Por lo tanto un **TDA es un tipo de dato compuesto**, que **genera un nuevo tipo de dato en el programa que lo define**.

En el lenguaje C existen 3 tipos de TDA:

- Estructuras, **struct**.
- Uniones, **union**.
- Enumeradores, **enum**.

[https://es.wikibooks.org/wiki/Programaci%C3%B3n en C/Estructuras y Uniones](https://es.wikibooks.org/wiki/Programaci%C3%B3n_en_C/Estructuras_y_Uniones)

Estructuras: Es un conjunto de tipos de datos primitivos y miembros (variables) que forman un tipo de dato compuesto.

- Definición

1) **struct** <nombre>{

<tipo de dato> <identificador1>; //Campo 1 o miembro 1.

<tipo de dato> <identificador2>; //Campo 2 o miembro 2.

...

<tipo de dato> <identificadorN>; //Campo N o miembro N.

};

struct <nombre> : Es el nuevo tipo de dato compuesto

2) typedef struct {

```
<tipo de dato> <identificador1>; //Campo 1 o miembro 1.  
<tipo de dato> <identificador2>; //Campo 2 o miembro 2.  
...  
<tipo de dato> <identificadorN>; //Campo N o miembro N.  
} <nombre>;
```

<nombre> : Es el nuevo tipo de dato compuesto.
--

1)

struct persona

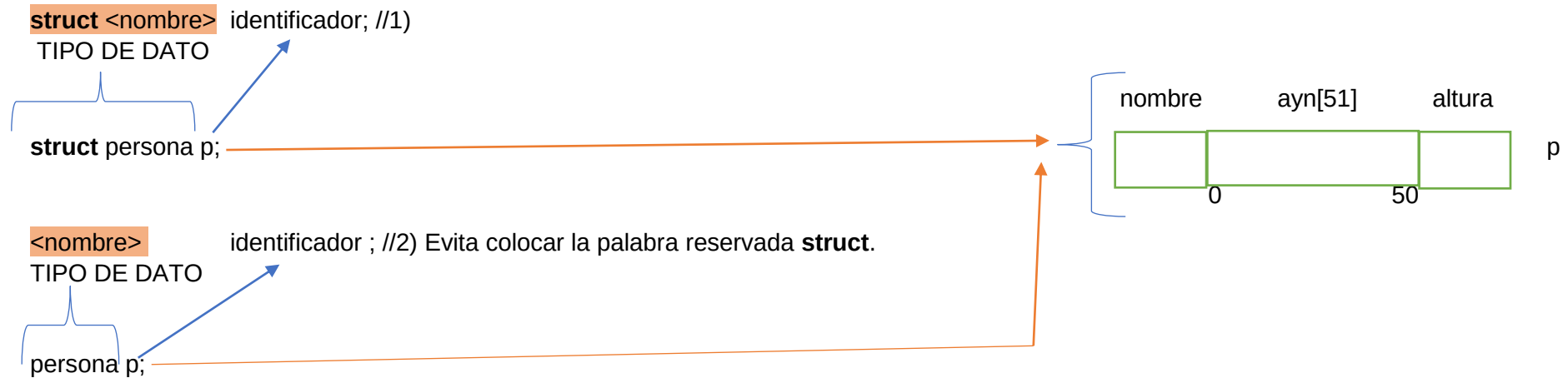
```
{  
    int legajo;  
    char ayn[51];  
    float altura;  
};
```

2)

typedef struct

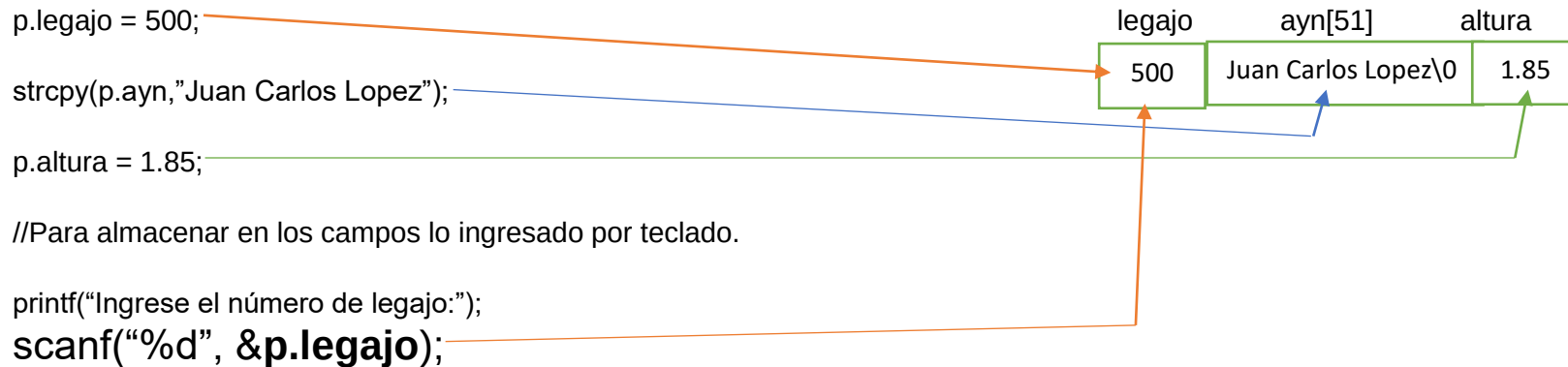
```
{  
    int legajo;  
    char ayn[51];  
    float altura;  
} persona;
```

- Declaración de un identificador de tipo estructura



- Acceso a los campos o miembros de una estructura: Se realiza a través del designador de campo (**caracter punto**).

Identificador .<nombre del campo o miembro>;



- Propiedades de las estructuras

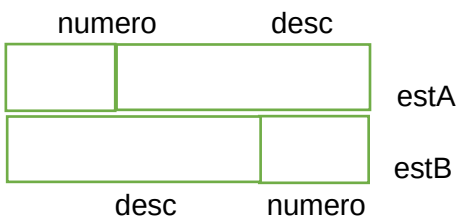
Asignación: Dos o más identificadores de tipo estructura se pueden asignar en forma directa, siempre y cuando sus tipos de dato pertenezcan a la misma estructura o sus tipos de datos pertenezcan a estructuras que coinciden en su cantidad de campos y tamaño en bytes, pero también deben coincidir en el orden de los tipos de datos de sus miembros (Condición de asignación de las estructuras).

Para obtener el tamaño en bytes de una estructura se utiliza la función **sizeof**(tipo de dato); que retorna la cantidad de bytes de un tipo de dato enviado como parámetro. Esta función es propia del lenguaje C y se encuentra en la biblioteca stdio.h

```
struct A{
    int numero; //4 bytes
    char desc[11]; //11 bytes
};           //total = 15 bytes
```

```
struct B{
    char desc[11]; //11 bytes
    int numero; //4 bytes.
           //total = 15 bytes
};
```

```
int main()
{
    struct A estA; //
    struct B estB; //
```



....

estB = estA; //ERROR al asignar. Coincide la cantidad de campos y de bytes, pero el orden de los tipos de datos es distinto.

```
}
```

Anidamiento: Dos o más estructuras se pueden anidar mediante la declaración un campo o miembro de una estructura en la otra estructura. El tipo de dato de este miembro es el nombre del tipo de dato compuesto.

```

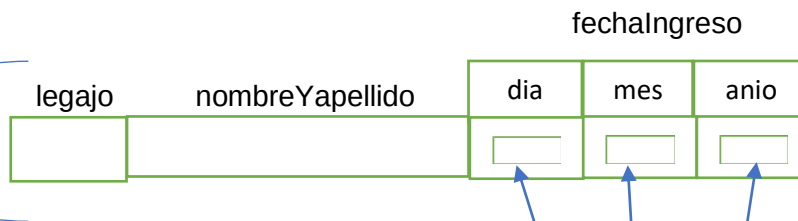
struct fecha {
    int dia;
    int mes;
    int anio;
};

struct persona {
    int legajo;
    char nombreYapellido[51];
    struct fecha fechalIngreso;
}
    
```

TIPO DE DATO CAMPO/MIEMBRO.

```
int main()
```

```
{
    struct persona p; //
```



```
printf("Cargar el día de ingreso:");
```

```
scanf ("%d", &p.fechalIngreso.dia ); //ACCESO AL MIEMBRO dia De struct fecha.
```

```
printf("Cargar el mes de ingreso:");
```

```
scanf ("%d", &p.fechalIngreso.mes ); //ACCESO AL MIEMBRO mes DE struct fecha
```

```
printf("Cargar el año de ingreso:");
```

```
scanf ("%d", &p.fechalIngreso.anio ); //ACCESO AL MIEMBRO anio DE struct fecha
```

```
..
return 0;
}
```

- Estructuras con Funciones

Parámetro real, actual o argumento: Si se desean enviar todos los campos de la estructura, sólo se coloca el identificador. Si sólo se desea enviar un miembro o más que no sean la totalidad, se coloca por cada miembro: identificador.nombreMiembro1, identificador.nombreMiembro2, ..., identificador.nombreMiembroN

```
MostrarPersona(p); //ENVIAR LA ESTRUCTURA COMPLETA.
```

```
MostrarNombreApellido (p.nombreYapellido); //ENVIAR UN SOLO CAMPO DE LA ESTRUCTURA.
```

Parámetro formal: Se coloca la declaración de una variable de tipo estructura para que reciba todos los campos de esta, sino se coloca un tipo de dato primitivo por cada tipo de dato del miembro de la estructura recibido.

```
void MostrarPersona (persona pa) // void MostrarPersona (struct persona pa) //RECIBE UN struct COMO PARÁMETRO FORMAL.  
{  
    ....  
}
```

Prototipo: Se coloca sólo el tipo de dato compuesto (**struct** <nombre> ó <nombre>);

```
void MostrarPersona (persona);// void MostrarPersona (struct persona)
```

Tipo de dato de retorno: Una función puede retornar un tipo de dato struct, al igual que un tipo de dato int, float, char, etc. En este caso la función retorna más de un valor.

```
persona CargaPersona(); //LA FUNCIÓN RETORNA UN TIPO DE DATO struct (persona).
```

```
int main()  
{  
    persona p;  
    p = CargaPersona();  
    ....  
  
    return 0;  
}
```

```

persona CargaPersona();
{
    persona pa;
    .....

    return pa;
}

```

Vectores de estructuras: “Heredan las mismas propiedades que los arrays unidimensionales” – Evitan la implementación de vectores paralelos -

Es un conjunto de variables de tipo estructura agrupada en forma continua y referenciadas bajo un nombre, cada variable pertenece a un tipo de dato compuesto.

- Declaración

```

struct <nombre> identificador [tamaño];

    <nombre> identificador [tamaño];

```

- Acceso

```

    identificador [índice].<nombre del campo o miembro>;

```

- Asignación: Se puede asignar una posición del vector de struct a una variable struct o viceversa, siempre y cuando cumplan con la restricción de asignación.

```

int main()
{
    persona per; //VARIABLE DE TIPO struct.
    persona vecper[5]; //VARIABLE DE TIPO vector de struct.

    per = CargaPersona();
    vec[0] = per; //ASIGNAR A LA POSICIÓN 0 DEL VECTOR EL CONTENIDO DE LA VARIABLE per.
    ....
    vec[1] = CargaPersona();
    per = vec[1]; //ASIGNAR A LA VARIABLE per EL CONTENIDO DE LA POSICIÓN 1 DEL VECTOR vec.
    ....
    return 0;

}

```



```
int main()
{
```

```
    struct persona vecPersona[5];
```

	legajo	nombreYapellido	fechaingreso			
			dia	mes	anio	
0						vecpersona[0]
1						vecpersona[1]
2						vecpersona[2]
3						vecpersona[3]
4						vecpersona[4]

```
    printf("Ingrese el número de legajo:");
```

```
    scanf ("%d", &vecpersona[0].legajo ); //ACCESO AL MIEMBRO legajo del índice 0
```

```
    printf("Ingrese el Nombre y apellido:");
```

```
    fflush(stdin);
```

```
    gets (vecpersona[0].nombreYapellido ); //ACCESO AL MIEMBRO nombreyapellido del índice 0
```

```
    printf("Cargar el día de ingreso:");
```

```
    scanf ("%d", & vecpersona[0].fechaIngreso.dia ); //ACCESO AL MIEMBRO DE struct fecha-del índice 0
```

```
    printf("Cargar el mes de ingreso:");
```

```
    scanf ("%d", & vecpersona[0].fechaIngreso.mes ); //ACCESO AL MIEMBRO DE struct fecha del índice 0
```

```
    printf("Cargar el año de ingreso:");
```

```
    scanf ("%d", & vecpersona[0].fechaIngreso.anio ); //ACCESO AL MIEMBRO DE struct fecha del índice 0
```

```
    ....
```

```
    return 0;
```

```
}
```

- Vectores de estructuras con Funciones

Parámetro real, actual o argumento: Si se desea enviar todo el array de la estructura, sólo se coloca el identificador. Si sólo se desea enviar un miembro o más que no sean la totalidad, se coloca por cada miembro: identificador[índice].nombreMiembro1, identificador[índice].nombreMiembro2, ..., identificador[índice].nombreMiembroN

CargaPersonas (vecPersona , 5); //ENVIAR TODO EL VECTOR DE struct.

MostrarLegajo (vecPersona[4].legajo); //ENVIAR COMO PARÁMETRO EL 5° LEGAJO.

Parámetro formal: Se coloca la declaración de un array de tipo estructura para que reciba todos los campos de esta, sino se coloca un tipo de dato primitivo del lenguaje (int, float, char) por cada tipo de dato del miembro de la estructura recibido.

```
void CargaPersonas (persona vecPersona[], int ce) // void CargaPersonas (struct persona vecPersona[], int ce)
{
    .....
}
```

```
void MostrarLegajo (int legajo)
{
    .....
}
```

Prototipo: Se coloca sólo el tipo de dato compuesto seguido de [] (**struct** <nombre>[] ó <nombre>[]);

void CargaPersonas (**persona** [], int); // void CargaPersonas (**struct persona** [], int);